

Design Patterns — Senior Developer Level

Design Pattern হলো কোনো নির্দিষ্ট code-এর copy-paste solution নয়।

বরং:

বারবার আসা **software design problem**-এর জন্য **proven structural approach**।

একটি pattern সাধারণত বলে:

```
Problem
↓
Design Idea
↓
Objects / Functions-এর Relationship
↓
Maintainable Solution
```

1. Design Pattern কেন দরকার?

ধরি একটি application-এ payment system আছে:

```
bKash
Nagad
Stripe
PayPal
```

যদি সব জায়গায়:

```
if (paymentType === "bkash") {
  // ...
} else if (paymentType === "nagad") {
  // ...
} else if (paymentType === "stripe") {
  // ...
}
```

লিখতে থাকো, application বড় হলে maintenance কঠিন হবে।

Design Pattern আমাদের সাহায্য করে:

```
Business Logic
  ↓
Stable Interface
  ↓
Different Implementations
```

ফলে নতুন implementation যোগ করা সহজ হয়।

2. Pattern শেখার আগে ৩টি প্রশ্ন

কোনো pattern দেখলে মুখস্থ না করে প্রথমে জিজ্ঞেস করবে:

প্রশ্ন ১

কোন **problem solve** করছে?

প্রশ্ন ২

কোন **objects/functions**-এর **relationship** পরিবর্তন হচ্ছে?

প্রশ্ন ৩

Pattern ব্যবহার করলে কী **trade-off** আসছে?

এই তিনটি বুঝলে pattern মনে রাখার দরকার অনেক কমে যায়।

3. Design Patterns-এর বড় Category

Classic GoF pattern-গুলোকে সাধারণভাবে:

```
Creational
  ↓
Object তৈরি করার সমস্যা

Structural
  ↓
Objects কীভাবে combine হবে

Behavioral
```

↓
Objects কীভাবে communicate করবে

এই section-এর patterns:

Singleton	→ Creational
Factory	→ Creational
Builder	→ Creational
Proxy	→ Structural
Decorator	→ Structural
Module	→ Structural
Observer	→ Behavioral
Strategy	→ Behavioral
Command	→ Behavioral
State Machine	→ Behavioral
Mediator	→ Behavioral
Repository	→ Architectural Pattern

4. Singleton Pattern

Problem

একটি application-এ কোনো resource-এর একটি **shared instance** দরকার।

যেমন:

- Database connection manager
- Logger
- Configuration
- Cache manager
- Application-wide service

Concept:

Request A ┘
Request B ┘ → Same Instance
Request C ┘

5. Basic Singleton

```
class Database {
  constructor() {
    if (Database.instance) {
      return Database.instance;
    }

    this.connection = null;

    Database.instance = this;
  }

  connect() {
    if (!this.connection) {
      this.connection = "Connected";
    }

    return this.connection;
  }
}

const db1 = new Database();
const db2 = new Database();

console.log(db1 === db2);
```

Result:

true

6. JavaScript Module Singleton

JavaScript-এ module caching-এর কারণে simple module নিজেই shared instance-এর মতো আচরণ করতে পারে।

```
class Logger {
  log(message) {
    console.log(message);
  }
}
```

```
const logger = new Logger();  
  
export default logger;
```

অন্য জায়গায়:

```
import logger from "./logger.js";
```

একই exported object ব্যবহার করা যায়।

7. Singleton-এর ভালো ব্যবহার

যেমন:

```
config  
logger  
metrics  
database manager  
cache manager
```

কিন্তু সবকিছুকে Singleton বানানো উচিত নয়।

8. Singleton-এর সমস্যা

Singleton technically global state-এর মতো আচরণ করতে পারে।

```
Any Module  
↓  
Global Singleton  
↓  
Shared Mutable State
```

এর ফলে:

- Testing কঠিন হতে পারে
- Hidden dependency তৈরি হয়
- State পরিবর্তনের source খুঁজে পাওয়া কঠিন
- Tight coupling হতে পারে

তাই:

Singleton useful, কিন্তু **default architecture** হিসেবে ব্যবহার করা উচিত নয়।

9. Factory Pattern

Factory-এর মূল idea:

Object তৈরির logic caller-এর কাছ থেকে আলাদা করে দেওয়া।

ধরি payment processor:

```
class BkashPayment {
  pay(amount) {
    console.log(`bKash: ${amount}`);
  }
}

class StripePayment {
  pay(amount) {
    console.log(`Stripe: ${amount}`);
  }
}
```

Factory:

```
function createPayment(type) {
  switch (type) {
    case "bkash":
      return new BkashPayment();

    case "stripe":
      return new StripePayment();

    default:
      throw new Error("Unsupported payment");
  }
}
```

ব্যবহার:

```
const payment = createPayment("bkash");  
  
payment.pay(500);
```

Caller-কে জানতে হচ্ছে না:

```
new BkashPayment()
```

কীভাবে তৈরি হয়।

10. Factory কেন useful?

Without Factory:

```
Business Logic  
↓  
new BkashPayment()  
new StripePayment()  
new NagadPayment()
```

Factory:

```
Business Logic  
↓  
createPayment()  
↓  
Implementation
```

Object creation-এর knowledge এক জায়গায় থাকে।

11. Abstract Factory

Factory সাধারণত একটি ধরনের object তৈরি করে।

Abstract Factory related **family of objects** তৈরি করে।

ধরি UI theme:

Dark Theme
├─ Button
└─ Input

Light Theme
├─ Button
└─ Input

Factory:

```
function createDarkUI() {  
  return {  
    createButton() {  
      return new DarkButton();  
    },  
  
    createInput() {  
      return new DarkInput();  
    }  
  };  
}
```

আর:

```
function createLightUI() {  
  return {  
    createButton() {  
      return new LightButton();  
    },  
  
    createInput() {  
      return new LightInput();  
    }  
  };  
}
```

12. Factory বনাম Abstract Factory

Factory
↓
এক ধরনের object

Abstract Factory
↓
Related object family

উদাহরণ:

Factory
→ Payment

Abstract Factory
→ Button + Input + Modal

13. Builder Pattern

Builder pattern useful যখন একটি object তৈরি করতে:

- অনেক options আছে
- কিছু optional
- construction step-by-step
- configuration complex

ধরি User:

```
const user = {  
  name: "Shariar",  
  email: "...",  
  age: 31,  
  role: "admin",  
  active: true  
};
```

অনেক parameter হলে constructor:

```
new User(  
  name,  
  email,  
  age,  
  role,  
  active,
```

```
...  
);
```

খুব ugly হয়ে যায়।

14. Builder Example

```
class UserBuilder {  
  constructor() {  
    this.user = {};  
  }  
  
  setName(name) {  
    this.user.name = name;  
    return this;  
  }  
  
  setEmail(email) {  
    this.user.email = email;  
    return this;  
  }  
  
  setRole(role) {  
    this.user.role = role;  
    return this;  
  }  
  
  setActive(active) {  
    this.user.active = active;  
    return this;  
  }  
  
  build() {  
    return this.user;  
  }  
}
```

ব্যবহার:

```
const user = new UserBuilder()  
  .setName("Shariar")  
  .setEmail("test@example.com")  
  .setRole("admin")
```

```
.setActive(true)
.build();
```

15. Builder-এর গুরুত্বপূর্ণ Concept: Fluent API

এই:

```
builder
    .setName(...)
    .setEmail(...)
    .setRole(...);
```

style-কে Fluent API বলা হয়।

কারণ প্রতিটি method সাধারণত:

```
return this;
```

করে।

16. Builder কোথায় useful?

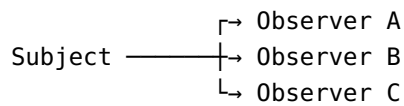
বিশেষ করে:

```
Complex configuration
Query builders
HTTP request builders
Test data builders
Configuration objects
SQL builders
```

17. Observer Pattern

Observer pattern-এর idea:

একটি object-এর state/event change হলে interested অন্য objects-কে notify করা।



Subject জানে না observer কী করছে।

সে শুধু notify করে।

18. Observer Example

```
class EventEmitter {
  constructor() {
    this.listeners = {};
  }

  on(event, callback) {
    if (!this.listeners[event]) {
      this.listeners[event] = [];
    }

    this.listeners[event].push(callback);
  }

  emit(event, data) {
    const callbacks = this.listeners[event] || [];

    for (const callback of callbacks) {
      callback(data);
    }
  }
}
```

ব্যবহার:

```
const emitter = new EventEmitter();

emitter.on("userCreated", user => {
  console.log("Send email:", user.email);
});

emitter.on("userCreated", user => {
  console.log("Create analytics event");
});
```

```
});  
  
emitter.emit("userCreated", {  
  id: 10,  
  email: "test@example.com"  
});
```

একটি event থেকে multiple subscriber trigger হচ্ছে।

19. Observer কোথায় ব্যবহার হয়?

Node.js:

EventEmitter

Frontend:

DOM Events

Application:

Event Bus
Pub/Sub
Notifications

Distributed system-এ conceptually:

```
Publisher  
  ↓  
Message Broker  
  ↓  
Subscribers
```

20. Observer-এর সমস্যা

অনেক subscriber থাকলে:

Event

↓

A

↓

B

↓

C

↓

D

কোন event কে trigger করছে এবং কে consume করছে বোঝা কঠিন হতে পারে।

এটাকে **event spaghetti** বলা যায়।

আর event listener remove না করলে memory leak-ও হতে পারে।

21. Strategy Pattern

Strategy pattern-এর idea:

একই কাজ করার একাধিক algorithm থাকলে algorithm-কে interchangeable করা।

ধরি shipping:

```
Inside Dhaka  
Outside Dhaka  
Express  
International
```

প্রতিটির calculation আলাদা।

22. Strategy Example

```
const insideDhaka = {  
  calculate(weight) {  
    return weight * 50;  
  }  
};  
  
const outsideDhaka = {
```

```
    calculate(weight) {  
        return weight * 100;  
    }  
};  
  
const express = {  
    calculate(weight) {  
        return weight * 200;  
    }  
};
```

Context:

```
function calculateShipping(strategy, weight) {  
    return strategy.calculate(weight);  
}
```

ব্যবহার:

```
calculateShipping(insideDhaka, 5);
```

অথবা:

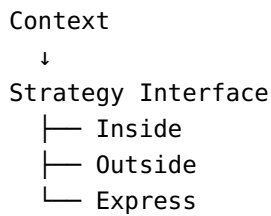
```
calculateShipping(express, 5);
```

23. Strategy-এর সবচেয়ে বড় সুবিধা

আপে:

```
if (type === "inside") {  
    ...  
} else if (type === "outside") {  
    ...  
} else if (type === "express") {  
    ...  
}
```

Strategy:



নতুন strategy যোগ করা সহজ।

24. Strategy কোথায় ব্যবহার হয়?

- Payment methods
- Authentication methods
- Sorting algorithms
- Pricing rules
- Discount calculation
- Shipping calculation
- Compression algorithms
- Serialization

25. Command Pattern

Command pattern:

কোনো operation/request-কে object/data হিসেবে represent করা।

Instead of:

```
editor.deleteText();
```

আমরা:

```
DeleteTextCommand
```

বানাতে পারি।

26. Command Example

```
class AddTextCommand {  
  constructor(editor, text) {  
    this.editor = editor;  
    this.text = text;  
  }  
  
  execute() {  
    this.editor.add(this.text);  
  }  
  
  undo() {  
    this.editor.remove(this.text);  
  }  
}
```

ব্যবহার:

```
const command =  
  new AddTextCommand(editor, "Hello");  
  
command.execute();  
  
command.undo();
```

এখানে operation-এর:

```
execute()  
undo()
```

দুটিই জানা আছে।

27. Undo/Redo System

Command pattern-এর classic use case:

```
Command 1  
Command 2
```

```
Command 3  
Command 4
```

History:

```
const history = [];
```

Execute:

```
history.push(command);  
command.execute();
```

Undo:

```
const command = history.pop();  
  
command.undo();
```

28. Command কোথায় ব্যবহার হয়?

- Text editor
- Undo/redo
- Game actions
- Job queues
- Transactions
- Audit logs
- Task scheduling

একটি গুরুত্বপূর্ণ idea:

Command হলো **behavior**-কে **data** হিসেবে **package** করা।

29. Proxy Pattern

Proxy pattern-এর idea:

একটি object-এর access/control-এর সামনে intermediary object বসানো।

```
Client
  ↓
Proxy
  ↓
Real Object
```

JavaScript-এর Proxy object এই pattern implement করার জন্য built-in mechanism দেয়।

30. Proxy Example — Access Control

```
const user = {
  name: "Sharlar",
  role: "user"
};

const proxyUser = new Proxy(user, {
  get(target, property) {
    if (property === "password") {
      throw new Error("Access denied");
    }

    return target[property];
  }
});
```

এখন:

```
proxyUser.name;
```

কাজ করবে।

কিন্তু:

```
proxyUser.password;
```

block করা যাবে।

31. Proxy-এর Practical Uses

- Validation
- Access control
- Logging
- Lazy loading
- Caching
- Reactive systems
- API abstraction
- Virtual objects

তবে JavaScript-এর **Proxy** এবং classic Proxy design pattern related হলেও পুরোপুরি একই concept নয়।

32. Decorator Pattern

Decorator pattern:

Existing object/class modify না করে তার behavior dynamically enhance করা।

```
Original Object
  ↓
Decorator
  ↓
More behavior
```

33. Logging Decorator Example

ধরি:

```
class UserService {
  createUser(data) {
    return {
      id: 10,
      ...data
    };
  }
}
```

Decorator:

```
function withLogging(service) {
  return {
    createUser(data) {
      console.log("Creating user");

      const result =
        service.createUser(data);

      console.log("User created");

      return result;
    }
  };
}
```

ব্যবহার:

```
const service =
  withLogging(new UserService());

service.createUser({
  name: "Shariar"
});
```

34. Multiple Decorators

```
UserService
  ↓
Logging
  ↓
Caching
  ↓
Authorization
  ↓
Metrics
```

যেমন:

```
const service =
  withMetrics(
    withCache(
```

```
        withLogging(  
            new UserService()  
        )  
    )  
};
```

এটা composition-এর শক্তি দেখায়।

35. Decorator কোথায় useful?

Backend-এ:

```
Logging  
Authentication  
Authorization  
Caching  
Metrics  
Retry  
Tracing
```

Frontend-এ:

```
Feature enhancement  
Instrumentation  
UI behavior
```

36. Decorator বনাম Proxy

দুটো কাছাকাছি মনে হতে পারে।

Decorator

মূলত:

```
Behavior add/extend
```

Proxy

মূলত:

Access/control/interception

সহজভাবে:

Decorator:

"Object-কে আরও capability দাও"

Proxy:

"Object-এর access-এর gatekeeper হও"

37. State Machine Pattern

State Machine খুব powerful যখন application-এর behavior state-এর উপর নির্ভর করে।

ধরি order:

Pending
Processing
Shipped
Delivered
Cancelled

সব state থেকে সব state-এ যাওয়া যাবে না।

38. State Transition

Pending
↓
Processing
↓
Shipped
↓
Delivered

অন্যদিকে:

Pending
↓
Cancelled

কিছু:

Delivered
↓
Pending

invalid |

39. State Machine Example

```
const transitions = {  
  pending: {  
    process: "processing",  
    cancel: "cancelled"  
  },  
  
  processing: {  
    ship: "shipped"  
  },  
  
  shipped: {  
    deliver: "delivered"  
  },  
  
  delivered: {},  
  
  cancelled: {}  
};
```

State transition:

```
function transition(state, event) {  
  const nextState =  
    transitions[state]?.[event];  
  
  if (!nextState) {  
    throw new Error(  

```

```
        `Invalid transition: ${state} -> ${event}`  
    );  
}  
  
    return nextState;  
}
```

ব্যবহার:

```
let state = "pending";  
  
state = transition(state, "process");  
  
state = transition(state, "ship");  
  
state = transition(state, "deliver");
```

শেষ:

delivered

40. State Machine কেন powerful?

Normal code:

```
if (status === "pending") {  
    ...  
}  
  
if (status === "processing") {  
    ...  
}  
  
if (status === "shipped") {  
    ...  
}
```

বড় হলে invalid state তৈরি হতে পারে।

State Machine explicit করে:

Current State

+

Event

↓

Next State

41. State Machine কোথায় ব্যবহার হয়?

বিশেষভাবে:

- Order processing
- Payment lifecycle
- Authentication
- Game logic
- UI workflow
- Upload process
- Job processing
- WebSocket connection lifecycle

তোমার Mafia/Werewolf game-এর মতো project-এও state machine খুব useful হতে পারে।

যেমন:

Lobby

↓

Game Starting

↓

Night

↓

Day

↓

Voting

↓

Result

↓

Night

এতে invalid transition prevent করা যায়।

42. Module Pattern — IIFE

ES Modules আসার আগে JavaScript-এ private state তৈরি করার জনপ্রিয় technique ছিল Module Pattern ।

মূলত:

```
IIFE  
+  
Closure
```

ব্যবহার করা হতো।

43. IIFE

IIFE:

```
(function () {  
  console.log("Executed");  
})();
```

Function define এবং immediately execute হচ্ছে।

44. Module Pattern Example

```
const counter = (function () {  
  let count = 0;  
  
  return {  
    increment() {  
      count++;  
    },  
  
    getCount() {  
      return count;  
    }  
  };  
})();
```

এখন:

```
counter.increment();  
  
console.log(counter.getCount());
```

কিন্তু:

```
counter.count;
```

undefined হবে।

কারণ `count` private।

45. Module Pattern কীভাবে কাজ করে?

```
IIFE  
├── Private variables  
└── Return public API  
    ├── increment()  
    └── getCount()
```

Closure private state ধরে রাখে।

46. Modern JavaScript-এ Module Pattern

আজকের JavaScript-এ:

```
export  
import
```

দিয়ে ES Modules আছে।

তাই IIFE Module Pattern অনেক ক্ষেত্রে পুরোনো approach।

কিন্তু concept এখনও important কারণ:

- Encapsulation
- Closures
- Private state
- Public API

বোঝার জন্য এটি fundamental।

47. Mediator Pattern

Mediator pattern:

অনেক objects একে অপরের সাথে directly communicate না করে একটি central mediator-এর মাধ্যমে communicate করবে।

Without mediator:

```
A ↔ B
A ↔ C
A ↔ D
B ↔ C
B ↔ D
C ↔ D
```

Complexity দ্রুত বাড়ে।

Mediator:

```
      A
      ↓
B → Mediator ← C
      ↑
      D
```

48. Mediator Example

```
class ChatMediator {
  constructor() {
    this.users = [];
```

```

    }

    addUser(user) {
      this.users.push(user);
    }

    send(sender, message) {
      for (const user of this.users) {
        if (user !== sender) {
          user.receive(message);
        }
      }
    }
  }
}

```

User:

```

class User {
  constructor(name, mediator) {
    this.name = name;
    this.mediator = mediator;
  }

  send(message) {
    this.mediator.send(this, message);
  }

  receive(message) {
    console.log(
      `${this.name} received: ${message}`
    );
  }
}

```

এখানে users একে অপরকে directly manage করছে না।

49. Middleware Pattern

Middleware concept Mediator-এর সাথে related।

Node.js/Express-style:

```
Request
  ↓
Middleware 1
  ↓
Middleware 2
  ↓
Middleware 3
  ↓
Controller
```

উদাহরণ:

```
function logger(req, res, next) {
  console.log(req.method, req.url);

  next();
}
```

Authentication:

```
function auth(req, res, next) {
  if (!req.user) {
    return res.status(401).json({
      message: "Unauthorized"
    });
  }

  next();
}
```

Pipeline:

```
app.use(logger);
app.use(auth);
```

50. Mediator বনাম Middleware

Mediator

মূলত:

Many objects
↓
Central communication coordinator

Middleware

মূলত:

Request
↓
Processing pipeline
↓
Next middleware

দুটোর common idea:

Components-কে loosely coupled রাখা।

51. Repository Pattern

Repository pattern বিশেষ করে backend architecture-এ খুব গুরুত্বপূর্ণ।

মূল idea:

Database access logic-কে business logic থেকে আলাদা করা।

Without Repository:

Controller
↓
SQL
↓
Business Logic
↓
SQL
↓
Response

সব mixed।

52. Repository Architecture

```
Controller
  ↓
Service
  ↓
Repository
  ↓
Database
```

Repository database-এর সাথে কথা বলে।

53. Repository Example

ধরি:

```
class UserRepository {
  async findById(id) {
    return db.query(
      "SELECT * FROM users WHERE id = ?",
      [id]
    );
  }

  async create(data) {
    return db.query(
      "INSERT INTO users ...",
      [data]
    );
  }
}
```

Service:

```
class UserService {
  constructor(userRepository) {
    this.userRepository = userRepository;
  }

  async getUser(id) {
    const user =
```

```
    await this.userRepository.findById(id);

    if (!user) {
      throw new Error("User not found");
    }

    return user;
  }
}
```

Controller:

```
async function getUser(req, res) {
  const user =
    await userService.getUser(req.params.id);

  res.json(user);
}
```

54. Repository-এর বড় সুবিধা

Business logic জানে না:

```
SQL
MongoDB
PostgreSQL
Prisma
Mongoose
```

Service শুধু জানে:

```
userRepository.findById()
userRepository.create()
```

Database implementation বদলালেও business layer কম পরিবর্তন করতে হয়।

55. Repository + Dependency Injection

আরও ভালো architecture:

```
class UserService {  
  constructor(repository) {  
    this.repository = repository;  
  }  
}
```

Production:

```
const service =  
  new UserService(realRepository);
```

Testing:

```
const fakeRepository = {  
  findById: async () => ({  
    id: 1,  
    name: "Test"  
  })  
};  
  
const service =  
  new UserService(fakeRepository);
```

এতে database ছাড়াই service test করা যায়।

56. Repository Pattern-এর গুরুত্বপূর্ণ Caveat

Repository সব project-এ দরকার নেই।

যদি application খুব ছোট হয়:

```
Controller  
↓  
ORM
```

হতেই পারে।

অপ্রয়োজনে repository layer যোগ করলে:

```
Controller
↓
Service
↓
Repository
↓
ORM
↓
DB
```

শুধু abstraction-এর জন্য abstraction তৈরি হতে পারে।

তাই:

Repository তখনই useful যখন data-access abstraction বাস্তব value দেয়।

57. Patternগুলোর মধ্যে সম্পর্ক

এখন সবচেয়ে গুরুত্বপূর্ণ অংশ।

এই pattern-গুলো isolated না।

একটি real backend application-এ একসাথে অনেক pattern থাকতে পারে।

```
Controller
↓
Mediator / Middleware
↓
Service
↓
Strategy
↓
Repository
↓
Database
```

Object creation:

Factory
↓
Service / Strategy

Cross-cutting behavior:

Decorator
↓
Logging / Cache / Metrics

State:

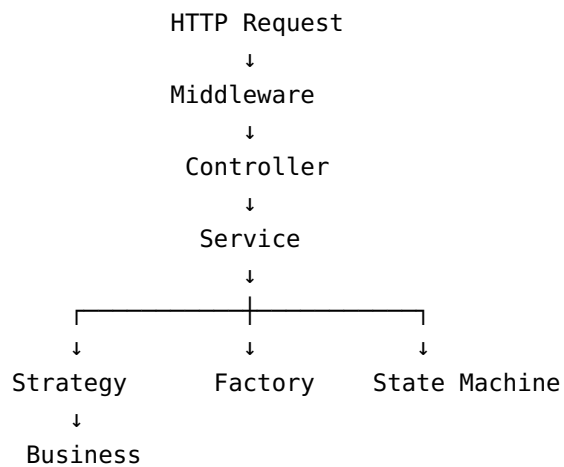
State Machine
↓
Order / Game / Payment

Events:

Observer
↓
EventEmitter

58. একটি Realistic Backend Architecture

ধরি একটি e-commerce backend।



```
Logic
↓
Repository
↓
Database
```

Events:

```
Order Created
↓
EventEmitter
├─ Send Email
├─ Update Analytics
└─ Notify User
```

Cross-cutting:

```
Service
↓
Logging Decorator
↓
Caching Decorator
↓
Metrics Decorator
```

এটাই pattern-এর real-world use ।

59. কোন Problem-এ কোন Pattern?

Problem	Pattern
একটি shared instance দরকার	Singleton
Object creation আলাদা করতে হবে	Factory
Related object family তৈরি	Abstract Factory
Complex object step-by-step তৈরি	Builder
Event-এর subscriber notify	Observer
Multiple interchangeable algorithm	Strategy
Operation-কে object/data করা	Command

Problem	Pattern
Object access intercept করা	Proxy
Existing behavior extend করা	Decorator
Complex state transition	State Machine
Private module state	Module
Central communication	Mediator
Request pipeline	Middleware
Data access abstraction	Repository

60. Singleton বনাম Factory

দুটো সম্পূর্ণ আলাদা problem solve করে।

Singleton

→ "কতগুলো instance থাকবে?"

Factory

→ "কোন object তৈরি হবে?"

61. Factory বনাম Builder

Factory

→ Which object?

Builder

→ How to construct complex object?

Factory:

```
createPayment("stripe");
```

Builder:

```
new UserBuilder()  
  .setName(...)  
  .setEmail(...)  
  .setRole(...)  
  .build();
```

62. Strategy বনাম State Machine

দুটোতেই behavior change হয়, কিন্তু উদ্দেশ্য আলাদা।

Strategy

Algorithm বদলায়:

```
Payment Strategy  
├─ Bkash  
├─ Stripe  
└─ Nagad
```

State Machine

Application-এর state এবং valid transition বদলায়:

```
Pending  
↓  
Processing  
↓  
Completed
```

সহজভাবে:

Strategy → "কীভাবে কাজ করব?"

State Machine → "এখন কোন state-এ আছি এবং next কী হতে পারে?"

63. Observer বনাম Mediator

Observer:

One event
↓
Many subscribers

Mediator:

Many components
↓
Central coordinator

Observer বেশি event-driven ।

Mediator বেশি communication coordination-oriented ।

64. Proxy বনাম Decorator

Proxy
→ Access control / interception

Decorator
→ Behavior enhancement

উদাহরণ:

Proxy:

User
↓
Permission Check
↓
User

Decorator:

Service
↓
Logging
↓
Caching

↓
Metrics

65. Command + State Machine

এরা একসাথে খুব powerful হতে পারে।

ধরি game:

PlayerMoveCommand
AttackCommand
DefendCommand

প্রতিটি command:

execute()

করবে।

State machine check করবে:

Current State
+
Command
↓
Allowed?

উদাহরণ:

Night
↓
AttackCommand
↓
Allowed

Day
↓
AttackCommand

↓
Rejected

66. Design Pattern Overuse

সবচেয়ে গুরুত্বপূর্ণ senior lesson:

Pattern দেখানোর জন্য **pattern** ব্যবহার করো না।

থারাপ:

```
Simple CRUD
↓
Factory
↓
Abstract Factory
↓
Builder
↓
Repository
↓
Mediator
↓
Decorator
```

এতে code unnecessarily complex হয়ে যাবে।

Pattern ব্যবহার করবে যখন:

```
Problem
↓
Repeated complexity
↓
Pattern reduces complexity
```

67. Pattern-এর আসল উদ্দেশ্য

Pattern-এর goal:

- ✗ Fancy code
- ✗ More classes
- ✗ More abstraction

- ✓ Lower coupling
- ✓ Better extensibility
- ✓ Clear responsibilities
- ✓ Easier testing
- ✓ Controlled complexity

68. SOLID-এর সাথে সম্পর্ক

Design Patterns বুঝতে SOLID জানা খুব helpful ।

S — Single Responsibility

একটি component-এর একটি primary responsibility ।

O — Open/Closed

নতুন behavior যোগ করতে existing code কম পরিবর্তন করা ।

Strategy/Decorator এতে সাহায্য করতে পারে ।

L — Liskov Substitution

Subtypes expected behavior maintain করবে ।

I — Interface Segregation

ছোট focused interface ।

D — Dependency Inversion

High-level logic low-level implementation-এর উপর সরাসরি depend না করা ।

Repository + Dependency Injection এখানে useful ।

69. Composition Over Inheritance

Modern JavaScript-এ একটি বড় design principle:

যেখানে সম্ভব inheritance-এর পরিবর্তে composition ব্যবহার করো।

Inheritance:

```
BaseService
  ↓
PaymentService
  ↓
AdvancedPaymentService
```

Composition:

```
PaymentService
+ Logger
+ Cache
+ Metrics
+ Authorization
```

Decorator, Strategy, Middleware—এগুলো composition-এর শক্তি দেখায়।

70. Design Patterns এবং Dependency Injection

Dependency Injection-এর basic idea:

```
class OrderService {
  constructor(repository, payment) {
    this.repository = repository;
    this.payment = payment;
  }
}
```

Service নিজে:

```
new PostgreSQLRepository()
```

বানাচ্ছে না।

তার dependency বাইরে থেকে দেওয়া হচ্ছে।

এতে:

```
Production
→ Real Repository

Testing
→ Fake Repository
```

দুটোই সহজ।

71. একটি Complete Example

ধরি Order System।

State Machine

```
Pending
↓
Paid
↓
Shipped
↓
Delivered
```

Strategy

```
PaymentStrategy
├─ Bkash
├─ Stripe
└─ Card
```

Repository

```
OrderService
↓
OrderRepository
↓
Database
```

Observer

```
OrderCreated
↓
├── Email
├── Notification
└── Analytics
```

Decorator

```
OrderService
↓
Logging
↓
Metrics
↓
Cache
```

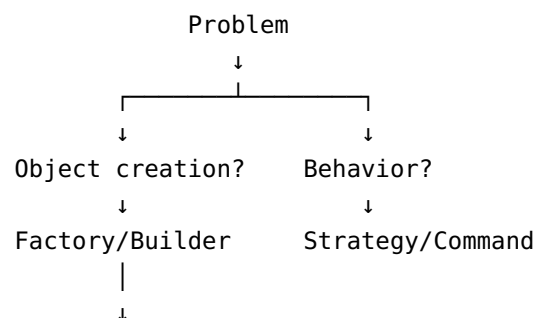
Factory

```
PaymentFactory
↓
PaymentStrategy
```

একটি real application-এ patternগুলো এভাবেই একে অপরের সাথে কাজ করে।

72. Senior Developer-এর জন্য সবচেয়ে গুরুত্বপূর্ণ Pattern Selection Flow

কোনো নতুন problem পেলে:



Access/Structure?



Proxy/Decorator



Communication?



Observer/Mediator



State complexity?



State Machine



Data access?



Repository

73. Quick Revision

Singleton

One shared instance

Factory

Object creation abstraction

Abstract Factory

Family of related objects

Builder

Complex object step-by-step construction

Observer

One event → many subscribers

Strategy

Interchangeable algorithms

Command

Operation → object

Proxy

Client → Proxy → Real Object

Decorator

Object → additional behavior

State Machine

State + Event → Next State

Module

Private state + public API

Mediator

Many components → central coordinator

Middleware

Request → processing pipeline

Repository

Business Logic → Data Access → DB

74. Interview Questions

Basic

1. Design Pattern কী?
2. Singleton কী?
3. Factory Pattern কেন ব্যবহার করা হয়?
4. Builder এবং Factory-এর difference কী?
5. Observer Pattern কী?
6. Strategy Pattern কী?

Intermediate

1. Strategy এবং State Machine-এর difference কী?
2. Decorator এবং Proxy-এর difference কী?
3. Observer এবং Mediator-এর difference কী?
4. Repository Pattern কেন useful?
5. Command Pattern দিয়ে Undo/Redo কীভাবে implement করা যায়?
6. JavaScript-এ Module Pattern কীভাবে কাজ করে?

Senior

1. Singleton কেন problematic হতে পারে?
2. কখন Factory ব্যবহার করা উচিত নয়?
3. Repository Pattern কি সব backend project-এ দরকার?
4. Composition over inheritance কেন গুরুত্বপূর্ণ?
5. Strategy Pattern কীভাবে Open/Closed Principle support করে?
6. Dependency Injection এবং Repository-এর relationship কী?
7. একটি event-driven architecture-এ Observer pattern-এর কী সমস্যা হতে পারে?
8. State Machine কখন `if/else` logic-এর চেয়ে ভালো?
9. একই application-এ Strategy + Factory + Repository কীভাবে combine করবে?
10. Decorator এবং Middleware-এর মধ্যে conceptual similarity কোথায়?

75. Practice Projects

Project 1 — Payment System

Implement:

```
PaymentStrategy
├── Bkash
├── Nagad
└── Stripe
```

এর জন্য:

```
Factory
+
Strategy
```

ব্যবহার করো।

Project 2 — Order System

State:

```
Pending
Paid
Shipped
Delivered
Cancelled
```

Implement:

```
State Machine
```

এবং invalid transition block করো।

Project 3 — Undo/Redo Editor

Implement:

Command
+
History Stack

Commands:

AddText
DeleteText
ReplaceText

প্রতিটির:

execute()
undo()

থাকবে।

Project 4 — Backend Service

Architecture:

Controller
↓
Middleware
↓
Service
↓
Repository
↓
Database

তারপর Service-এ:

Decorator
→ Logging
→ Metrics

যোগ করো।

Project 5 — Event-driven Order

Order create হলে:

```
OrderCreated
  ↓
EventEmitter
  ├── Send Email
  ├── Update Analytics
  └── Send Notification
```

Implement করো।

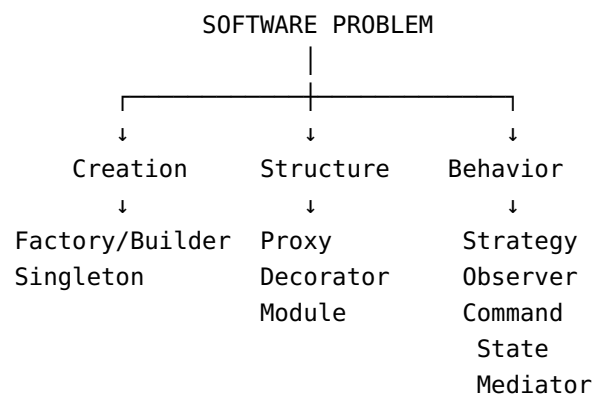
76. Final Mental Model

Design Pattern-কে কখনো:

"১২টা pattern মুখস্থ করতে হবে"

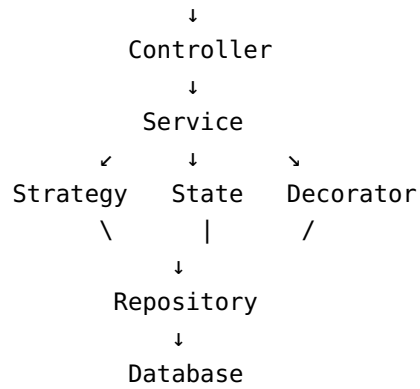
এইভাবে দেখবে না।

বরং:



আর architecture level-এ:

```
Application
  |
Middleware
```



সবচেয়ে গুরুত্বপূর্ণ senior-level rule:

Design Pattern হলো **solution**-এর উদ্দেশ্য নয়; **problem solve** করার **tool**।

আর একটি **pattern** ব্যবহার করার আগে সবসময় ভাববে:

Does this reduce complexity?

↓

Yes

↓

Use it.

No

↓

Don't use it.

শেষ পর্যন্ত একজন senior developer-এর skill হলো কতগুলো **pattern** জানে সেটা নয়; বরং কখন **pattern** ব্যবহার না করাই ভালো সেটাও বুঝতে পারে।